
MODULE 01: Python Fundamentals and Basic Data Structures

1. Designer Door Mat

This program generates a symmetrical pattern, typically used to simulate a door mat design, with a "WELCOME" message in the center.

Line	Code	Explanation
1	<code>n, m = map(int,input().split())</code>	Input Dimensions: Reads two space-separated integers from the user. <code>input().split()</code> splits the line into a list of strings. <code>map(int, ...)</code> converts those strings to integers. <code>n</code> is the height (must be an odd number), and <code>m</code> is the width (must be $3 \times n$).
2	<code>`pattern = [('.</code>	<code>.(2i + 1)).center(m, '-') for i in range(n//2)]`</code>
3	<code>print('\n'.join(pattern + ['WELCOME'.center(m, '-')] + pattern[::-1]))</code>	Print Full Mat: Joins the parts to print the final output. · <code>['WELCOME'.center(m, '-')]</code> creates the center line, centering the 'WELCOME' text with hyphens. · <code>pattern[::-1]</code> creates the bottom half by reversing the pattern list (the top half). · <code>pattern + [center line] + pattern[::-1]</code> concatenates the top, center, and bottom lists. · <code>'\n'.join(...)</code> joins all the lines in the final list with a newline character (<code>\n</code>) in between, printing the complete pattern.

Desired Input/Output Example:

- **Input:** 7 21 (Height $n = 7$, Width $m = 21$)
- **Output:**

```
-----|.-----
-----|.|.|.-----
----|.|.|.|.|.----
----WELCOME-----
----|.|.|.|.|.----
-----|.|.|.-----
-----|.-----
```

2. Find a String

This program counts how many times a given sub_string appears within a larger string.

Line	Code	Explanation
1	def count_substring(string, sub_string):	Defines a function named count_substring that takes two arguments: the main string and the sub_string to search for.
2	count = 0	Initializes a variable count to zero. This will store the number of occurrences found.
3	for i in range(len(string)):	Starts a loop that iterates through every possible starting index i of the main string.
4	if string[i:].startswith(sub_string):	Checks for a match: · string[i:] creates a slice of the main string starting from the index i until the end. · .startswith(sub_string) checks if this sliced portion begins with the sub_string. This approach ensures overlapping occurrences are counted (e.g., 'ABA' in 'ABABABA').
5	count += 1	If the condition on line 4 is true (a match is found), the count is incremented by 1.
6	return count	After checking all possible starting positions, the function returns the final count.
7	if __name__ == '__main__':	Standard Python block to ensure the code inside only runs when the script is executed directly, not when imported as a module.
8	string = input().strip()	Reads the main string from the user and removes any leading/trailing whitespace using .strip().
9	sub_string = input().strip()	Reads the sub-string from the user and removes any leading/trailing whitespace.
10	count = count_substring(string, sub_string)	Calls the function with the two inputs to get the result.
11	print(count)	Prints the final count of occurrences.

Desired Input/Output Example:

- **Input (main string):** ABCDCDC
 - **Input (sub-string):** CDC
 - **Output:** 2
 - (The first 'CDC' starts at index 2, the second 'CDC' starts at index 4).
-

3. Swap Case

This program iterates through a string and converts every uppercase letter to lowercase and every lowercase letter to uppercase, leaving other characters (like numbers or symbols) unchanged.

Line	Code	Explanation
1	def swap_case(s):	Defines a function named swap_case that takes one string argument s.
2	res = ""	Initializes an empty string res (for result) to build the new string.
3	for c in s:	Starts a loop that iterates over each character c in the input string s.
4	if c.isalpha and c.islower():	Check 1: Lowercase letter. Checks if c is an alphabet character (c.isalpha()) and is lowercase (c.islower()). Note: The c.isalpha check here is redundant/incomplete and should be c.isalpha() (a method call), but Python's boolean evaluation often handles it (or it's implied by islower()). The core logic is to check if it's lowercase.
5	res += c.upper()	If the character is lowercase, it is converted to uppercase using .upper() and appended to the result string res.
6	elif c.isalpha and c.isupper():	Check 2: Uppercase letter. Checks if c is an alphabet character and is uppercase (c.isupper() should be used here instead of c.isupper()). Again, assuming the intent is to check for uppercase.
7	res += c.lower()	If the character is uppercase, it is converted to lowercase using .lower() and appended to the result string res.
8	else:	If the character is neither a lowercase nor an uppercase letter (e.g., number, space, symbol).
9	res += c	The character is appended to res unchanged.
10	return res	After the loop finishes, the modified string res is returned.
11	if __name__ == '__main__':	Standard execution block.
12	s = input()	Reads the input string from the user.
13	result = swap_case(s)	Calls the function to get the swapped-case string.
14	print(result)	Prints the final result.

Note on a better/simpler implementation: Python strings have a built-in method s.swapcase(), which performs this exact function directly and more efficiently: return s.swapcase().

Desired Input/Output Example:

- **Input:** Hello World 123
- **Output:** hELLO wORLD 123

4. Nested Lists

This program processes a list of student names and their scores, then finds the name(s) of the student(s) who have the **second lowest score**.

Line	Code	Explanation
1	<code>if __name__ == '__main__':</code>	Standard execution block.
2	<code># Step 1: Take input</code>	Comment indicating the input section.
3	<code>students = []</code>	Initializes an empty list named students to store the name/score pairs.
4	<code>for _ in range(int(input("Enter number of students: "))):</code>	Reads the number of students, converts it to an integer, and loops that many times. <code>_</code> is a convention for a variable that won't be used inside the loop.
5	<code>name = input("Enter name: ")</code>	Reads the student's name.
6	<code>score = float(input("Enter score: "))</code>	Reads the student's score and converts it to a floating-point number.
7	<code>students.append([name, score])</code>	Appends a list containing the [name, score] pair to the students list.
8	<code># Step 2: Extract all scores</code>	Comment indicating the score extraction step.
9	<code>scores = [s for name, s in students]</code>	List Comprehension: Creates a new list scores containing <i>only</i> the score from each [name, score] pair in the students list.
10	<code># Step 3: Find the second lowest score</code>	Comment indicating the calculation step.
11	<code>second_lowest = sorted(set(scores))[1]</code>	The Core Logic: · <code>set(scores)</code> removes duplicate scores (ensuring unique values). · <code>sorted(...)</code> sorts the unique scores in ascending order. · <code>[1]</code> accesses the element at index 1 of the sorted list, which is the second smallest unique score.
12	<code># Step 4: Print names with that score (sorted alphabetically)</code>	Comment indicating the output step.

Line	Code	Explanation
13	for name, score in sorted(students):	Loops through the students list. Crucially, sorted(students) sorts the list of lists. When sorting lists of lists, Python sorts by the first element (name), which ensures the output names are alphabetical.
14	if score == second_lowest:	Checks if the current student's score matches the second_lowest score found earlier.
15	print(name)	If the scores match, the student's name is printed.

Desired Input/Output Example:

- **Input (Number of students):** 5
- **Input (Name/Score pairs):** Harry/37.21, Berry/37.21, Tina/37.2, Akriti/41, Harsh/39
- **Unique Scores:** 37.2,37.21,39,41
- **Sorted Unique Scores:** [37.2, **37.21**, 39,41]
- **Second Lowest Score:** 37.21
- **Output:** (Names with 37.21, sorted alphabetically)

Berry

Harry

5. List Comprehensions

This program uses a concise **list comprehension** to generate a list of all possible coordinates $[i, j, k]$ from a 3D grid, where the sum of the coordinates ($i + j + k$) is **not** equal to a given number n .

Line	Code	Explanation
1	if __name__ == '__main__':	Standard execution block.
2	x = int(input())	Reads the integer for the max value of the first dimension (i).
3	y = int(input())	Reads the integer for the max value of the second dimension (j).
4	z = int(input())	Reads the integer for the max value of the third dimension (k).
5	n = int(input())	Reads the integer to be avoided for the sum ($i + j + k$).

Line	Code	Explanation
6	<pre>print([[i,j,k] for i in range(x+1) for j in range(y+1) for k in range(z+1) if(i+j+k) != n])</pre>	The List Comprehension: ·[i,j,k] is the output format for each element (a list of 3 coordinates). ·for i in range(x+1): Loops for $i = 0$ up to x. ·for j in range(y+1): Loops for $j = 0$ up to y. ·for k in range(z+1): Loops for $k = 0$ up to z. ·if(i+j+k) != n: This is the condition; the list element is only included if the sum of the coordinates is <i>not</i> equal to n.

Desired Input/Output Example:

- **Input (x):** 1
- **Input (y):** 1
- **Input (z):** 1
- **Input (n):** 2
- **Possible Coordinates (i from 0 to 1, j from 0 to 1, k from 0 to 1):**
 - [0,0,0](Sum 0)
 - [0,0,1](Sum 1)
 - [0,1,0](Sum 1)
 - [0,1,1](Sum 2, Excluded)
 - [1,0,0](Sum 1)
 - [1,0,1](Sum 2, Excluded)
 - [1,1,0](Sum 2, Excluded)
 - [1,1,1](Sum 3)
- **Output:** [[0, 0, 0], [0, 0, 1], [0, 1, 0], [1, 0, 0], [1, 1, 1]]

MODULE 02: Strings, Collections, and Date/Time

1. Alphabet Rangoli

This program prints an "Alphabet Rangoli" (a geometric pattern) of size n . The pattern is made of hyphens and lowercase English letters, starting with the n -th letter from the end (e.g., if $n = 5$, it starts with 'e').

Line	Code	Explanation
1	import string	Imports the built-in string module, which contains useful string constants.
2	def print_rangoli(n):	Defines the function print_rangoli with size n.
3	alpha = string.ascii_lowercase	Stores the string 'abcdefghijklmnopqrstuvwxyz' in the variable alpha.
4	L = []	Initializes an empty list L to store the lines of the top half of the rangoli.
5	for i in range(n):	Loops from $i = 0$ up to $n - 1$. Each iteration generates one line.
6	s = "-".join(alpha[i:n])	Generates the right part of the line: · alpha[i:n] takes a slice of letters from the i -th letter up to the n -th letter (exclusive). E.g., if $n = 5$, for $i = 0$, slice is 'abcde'; for $i = 1$, 'bcde'. · "-".join(...) joins these letters with hyphens. E.g., 'a-b-c-d-e'.
7	L.append((s[::-1]+s[1:]).center(4*n-3, "-"))	Generates and stores the full line: · s[::-1] reverses the string s (e.g., 'e-d-c-b-a'). · s[1:] takes the string s <i>starting from the second character</i> (to avoid repeating the center letter). E.g., 'b-c-d-e'. · s[::-1]+s[1:] concatenates them to form the full symmetric string (e.g., 'e-d-c-b-a-b-c-d-e'). · .center(4*n-3, "-") centers this string within the total width of the rangoli (which is $4n - 3$), padding with hyphens. The line is then appended to list L.
8	print("\n".join(L[:0:-1]+L))	Prints the final pattern: · L[:0:-1] is a slice that reverses the list L but excludes the element at index 0 (the center line), forming the bottom half's lines <i>above</i> the center line. · L is the full top half (including the center line). · L[:0:-1] + L concatenates the bottom half and the top half (which includes the center line, making it the complete rangoli). · '\n'.join(...) prints the final output.
9	if __name__ == '__main__':	Standard execution block.
10	n = int(input())	Reads the size n from the user.
11	print_rangoli(n)	Calls the function to print the rangoli.

Desired Input/Output Example:

- **Input:** 3 (Letters c, b, a)
- **Output:**

```

----c----
--c-b-c--
c-b-a-b-c
--c-b-c--
----c----

```

2. String Validators

This program takes an input string and checks five properties: whether it contains any alphanumeric characters, any alphabetical characters, any digits, any lowercase characters, and any uppercase characters.

Line	Code	Explanation
1	if __name__ == '__main__':	Standard execution block.
2	s = input()	Reads the input string from the user.
3	print(any(c.isalnum() for c in s))	Alphanumeric check: ·c.isalnum() checks if a character c is a letter or a number. ·The generator expression (c.isalnum() for c in s) generates True/False for every character. ·any(...) returns True if at least one of the values in the generator is True (i.e., if the string contains <i>any</i> alphanumeric character).
4	print(any(c.isalpha() for c in s))	Alphabetical check: Checks if the string contains <i>any</i> letter (a-z, A-Z).
5	print(any(c.isdigit() for c in s))	Digit check: Checks if the string contains <i>any</i> digit (0-9).
6	print(any(c.islower() for c in s))	Lowercase check: Checks if the string contains <i>any</i> lowercase letter.
7	print(any(c.isupper() for c in s))	Uppercase check: Checks if the string contains <i>any</i> uppercase letter.

Desired Input/Output Example:

- **Input:** aC1
- **Output:**

True (because 'a', 'C', and '1' are alphanumeric)

True (because 'a' and 'C' are alphabetic)

True (because '1' is a digit)

True (because 'a' is lowercase)

True (because 'C' is uppercase)

3. The Captain's Room

This program is designed to find the **unique room number** (The Captain's Room number) in a list where every other room number is repeated a certain number of times (k).

Mathematical Insight: If k is the number of times every room is repeated *except* the Captain's room (which is repeated only once), then:

1. **Sum of Unique Rooms $\times k$:** This sum overcounts the Captain's room by $k - 1$ times.
2. **Sum of All Rooms:** This is the actual sum of the list.
3. **Difference:** (Sum of Unique $\times k$) $-$ (Sum of All) equals the Captain's Room number repeated $k - 1$ times (since the other room numbers cancel out).
4. **Result:** Divide the difference by $k - 1$ to get the Captain's Room number.

Line	Code	Explanation
1	<code>k = int(input())</code>	Reads the integer k , which is the number of times the normal room numbers are repeated.
2	<code>lst = list(map(int, input().split()))</code>	Reads the list of all room numbers from a single line of space-separated integers.
3	<code>captain = (sum(set(lst)) * k - sum(lst)) // (k - 1)</code>	The Core Calculation: ·<code>set(lst)</code>: Creates a set of unique room numbers. ·<code>sum(set(lst))</code>: Calculates the sum of all unique room numbers. ·<code>sum(set(lst)) * k</code>: Calculates the sum if <i>every</i> unique room number appeared k times. ·<code>sum(lst)</code>: Calculates the actual total sum of the entire input list. ·The difference (Sum Unique $\times k -$ Sum All) is the Captain's room number multiplied by $(k - 1)$. ·<code>// (k - 1)</code>: Performs integer division by $k - 1$ to isolate the unique Captain's room number.
4	<code>print(captain)</code>	Prints the final result.

Desired Input/Output Example:

- **Input (k):** 3
- **Input (List):** 1 2 3 6 5 4 4 2 5 3 6 1 6 5 3 2 4 1 2 5 1 3 4 6
- **Unique Rooms:** 1,2,3,4,5,6

- Captain's Room:** 6 is repeated 4 times, 1,2,3,4,5 are repeated 3 times. **Wait, the problem statement usually implies the Captain's room is the *only* one repeated only once (or a different number of times than k). Assuming $k = 3$ means all rooms *except* the captain's are repeated 3 times, and the Captain's room is repeated once.** Let's follow the standard problem constraint: $k = 3$, all rooms repeated 3 times, Captain's room repeated once.
 - Actual List (Example where 6 is the Captain):** 1 1 1 2 2 2 3 3 3 4 4 4 5 5 5 6 (All 1 – 5 are 3 times, 6 is 1 time)
 - Sum of Unique $\times k$ (Hypothetical):** $(1 + 2 + 3 + 4 + 5 + 6) \times 3 = 21 \times 3 = 63$
 - Sum of All (Actual):** $(1 \times 3 + 2 \times 3 + 3 \times 3 + 4 \times 3 + 5 \times 3 + 6 \times 1) = 3 + 6 + 9 + 12 + 15 + 6 = 51$
 - Difference:** $63 - 51 = 12$
 - Result:** $12 // (3 - 1) = 12 // 2 = 6$
- Output (Based on the logic):** 6

4. Time Delta

This program calculates the absolute difference in seconds between two given timestamps, which include time zone information.

Line	Code	Explanation
1	import math...	Imports necessary system modules (some may not be strictly required for the function itself but are common boilerplate).
6	from datetime import datetime	Imports the datetime class from the datetime module, essential for handling timestamps.
7	def time_delta(t1, t2):	Defines the function time_delta taking two timestamp strings, t1 and t2.
8	fmt = "%a %d %b %Y %H:%M:%S %z"	Defines the format string that matches the structure of the input timestamps. E.g., %a is weekday (Mon), %d is day (27), %b is month (Feb), %Y is year (2020), %H:%M:%S is time (10:00:00), and %z is the time zone offset (+0530).
9	dt1 = datetime.strptime(t1,fmt)	Parses the first string t1 into a datetime object using the defined fmt.
10	dt2 = datetime.strptime(t2,fmt)	Parses the second string t2 into a datetime object.
11	k = (abs(dt1-dt2).total_seconds())	Calculates the difference: ·dt1 - dt2: Subtracts the two datetime objects, resulting in a timedelta object (a duration).

Line	Code	Explanation
		<code>abs(...)</code> : Takes the absolute value of the timedelta (ensuring the result is positive). <code>.total_seconds()</code> : Converts the duration (timedelta) into the total number of seconds.
12	<code>return str(int(k))</code>	Converts the total seconds (a float) to an integer, then to a string, and returns it.
13	<code>if __name__ == '__main__':</code>	Standard execution block.
14	<code>fptr = open(os.environ['OUTPUT_PATH'], 'w')</code>	File handling boilerplate (common in competitive programming platforms) to open the output file.
15	<code>t = int(input())</code>	Reads the number of test cases.
16	<code>for t_itr in range(t):</code>	Loops through all test cases.
17	<code>t1 = input()</code>	Reads the first timestamp string for the current test case.
18	<code>t2 = input()</code>	Reads the second timestamp string.
19	<code>delta = time_delta(t1, t2)</code>	Calls the function to get the difference.
20	<code>fptr.write(delta + '\n')</code>	Writes the result to the output file.
21	<code>fptr.close()</code>	Closes the output file.

Desired Input/Output Example:

- **Input (t1):** Sun 10 May 2015 13:54:36 -0700
- **Input (t2):** Sun 10 May 2015 13:54:36 +0000
- **Explanation:** The absolute time difference is solely due to the timezone offset (0700 → 0000), which is 7 hours.
- **Calculation:** 7 hours × 60 min/hour × 60 sec/min = 25200 seconds.
- **Output:** 25200

5. Map and lambda function

This program calculates the cube of the first n Fibonacci numbers. It uses a custom fibonacci function, a lambda function for cubing, and the built-in map function.

Line	Code	Explanation
1	<code>cube = lambda x:x ** 3</code>	Lambda Function: Defines a small, anonymous function named <code>cube</code> that takes one argument <code>x</code> and returns <code>x</code> cubed (x^3).
2	<code>def fibonacci(n):</code>	Defines a function to generate the first <code>n</code> Fibonacci numbers.
3	<code># return a list of fibonacci numbers</code>	Comment.
4	<code>fib_list=[]</code>	Initializes an empty list to store the Fibonacci numbers.
5	<code>a,b =0,1</code>	Initializes the first two numbers in the Fibonacci sequence: $a = 0$ and $b = 1$.
6	<code>for _ in range(n):</code>	Loops <code>n</code> times to generate <code>n</code> numbers.
7	<code>fib_list.append(a)</code>	Appends the current value of <code>a</code> (the next Fibonacci number) to the list.
8	<code>a,b=b,a+b</code>	Fibonacci Step: Simultaneously updates <code>a</code> to the old value of <code>b</code> , and <code>b</code> to the sum of the old <code>a</code> and <code>b</code> ($a + b$). This moves the sequence forward: $(0,1) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (2,3) \rightarrow \dots$
9	<code>return fib_list</code>	Returns the list of the first <code>n</code> Fibonacci numbers.
10	<code>if __name__ == '__main__':</code>	Standard execution block.
11	<code>n = int(input())</code>	Reads the count <code>n</code> from the user.
12	<code>print(list(map(cube, fibonacci(n))))</code>	Map and Print: · <code>fibonacci(n)</code> : Generates the list of <code>n</code> Fibonacci numbers (the iterable). · <code>map(cube, ...)</code> : Applies the <code>cube</code> function to <i>every element</i> in the Fibonacci list, producing a map object (an iterator of cubed numbers). · <code>list(...)</code> : Converts the map object into a final list. · <code>print(...)</code> : Prints the final list of cubed Fibonacci numbers.

Desired Input/Output Example:

- **Input (n):** 5
- **Fibonacci Numbers (n=5):** [0,1,1,2,3]
- **Cubed Numbers:** $[0^3, 1^3, 1^3, 2^3, 3^3] = [0,1,1,8,27]$
- **Output:** [0, 1, 1, 8, 27]

MODULE 03: Advanced Collections and Regular Expressions

2. Validating CC Numbers

This program checks if a credit card number string is valid based on a set of rules, primarily using a **Regular Expression (regex)**.

Rules for Validity:

1. Must start with 4, 5, or 6.
2. Must contain exactly 16 digits (can be grouped as 4 × 4 with optional hyphens).
3. May have hyphens (-) separating groups of 4 digits, e.g., 4000-0000-0000-0000.
4. Must NOT contain 4 or more consecutive identical digits (even if separated by hyphens).

Line	Code	Explanation
4	import re	Imports the re module for Regular Expression operations.
5	TESTER = re.compile(	Compiles the main regular expression pattern into a TESTER object for efficiency.
6	r"^"	Start Anchor: Asserts the pattern must match from the beginning of the string.
7	r"(?!.*(\d)(-?\1){3})"	Negative Lookahead: The most complex part. It asserts that at any position (.*?) in the string, you cannot find: <code>·(\d)</code> : A digit (captured as group 1). <code>·(-?\1){3}</code> : This digit repeated 3 more times, where each repeat may be preceded by an optional hyphen (-?). This enforces Rule 4 (no four consecutive identical digits, even with hyphens).
8	r"[456]"	Start Digit: Must start with a 4, 5, or 6 (enforces Rule 1).
9	r"\d{3}"	First Group: Matches exactly 3 more digits. (First 4 digits matched).
10	r"(?:-?\d{4}){3}"	Remaining Groups: Matches the remaining three 4-digit groups. <code>·(?:...)</code> : Non-capturing group. <code>·-?</code> : Optional hyphen. <code>·\d{4}</code> : Exactly 4 digits. <code>·{3}</code> : This entire pattern repeats exactly 3 times.
11	r"\$"	End Anchor: Asserts the pattern must match up to the end of the string.
13	for _ in range(int(input().strip())):	Reads the number of test cases and loops.

Line	Code	Explanation
14	<pre>print("Valid" if TESTER.search(input().strip()) else "Invalid")</pre>	Validation and Output: ·input().strip(): Reads the credit card number. ·TESTER.search(...): Attempts to find a match for the full regex pattern in the input. ·The result is printed as "Valid" if a match is found, and "Invalid" otherwise.

Desired Input/Output Example:

- **Input (Count):** 2
- **Input 1:** 4123456789012345 (Starts with 4, 16 digits, no repeat of 4)
- **Input 2:** 5123-3823-3000-0000 (Starts with 5, 16 digits, but 0000 violates Rule 4)
- **Output:**

Valid

Invalid

3. No Idea !

This program calculates a person's happiness score based on a list of numbers. The score increases by 1 for every number in the list that is present in a "happy set" (A) and decreases by 1 for every number in the list that is present in a "sad set" (B).

Line	Code	Explanation
1	<pre>n, m = map(int, input().split())</pre>	Reads two integers (n , m) from the first line. n is the length of the list, and m is the size of sets A and B .
2	<pre>lst = list(map(int, input().split()))</pre>	Reads the second line and stores the main list of numbers in <code>lst</code> .
3	<pre>A, B = set(map(int, input().split())), set(map(int, input().split()))</pre>	Reads the next two lines and converts them into sets A and B . Using sets makes membership checking (i.e., i in A) extremely fast.
4	<pre>print(sum((i in A) - (i in B) for i in lst))</pre>	The Core Calculation (Generator Expression): ·for i in <code>lst</code>: Iterates through every number i in the main list. ·$(i$ in $A)$: Checks if i is in set A. This boolean expression is treated as an integer: True = 1, False = 0. ·$(i$ in $B)$: Checks if i is in set B (True = 1, False = 0). ·$(i$ in $A) - (i$ in $B)$: Calculates the score for that number i. If i is in A (and not B), score is $1 - 0 = 1$. If i is in B (and not A), score is $0 - 1 = -1$. If it's in neither, $0 - 0 = 0$. If it's in both, $1 - 1 = 0$.

Line	Code	Explanation
		·sum(...): Adds up the scores for all numbers in lst to get the final total happiness.

Desired Input/Output Example:

- **Input (n, m):** 3 2
- **Input (List):** 1 5 3
- **Input (Set A):** 3 1
- **Input (Set B):** 5 7
- **Happiness Score Breakdown:**
 - For 1: In $A(1)$, not in $B(0)$. Score: $1 - 0 = +1$
 - For 5: Not in $A(0)$, in $B(1)$. Score: $0 - 1 = -1$
 - For 3: In $A(1)$, not in $B(0)$. Score: $1 - 0 = +1$
 - Total: $1 + (-1) + 1 = 1$
- **Output:** 1

4. Word Order

This program counts the frequency of words in an input list and prints the **number of unique words** and the **frequency** of each word, maintaining the order of their first appearance.

Line	Code	Explanation
1	from collections import Counter, OrderedDict	Imports Counter for easy counting and OrderedDict to maintain insertion order.
2	class OrderedCounter(Counter, OrderedDict):	Custom Class: Creates a new class that inherits from both Counter (for counting) and OrderedDict (to ensure the keys/words remain in the order they were first encountered).
4	d = OrderedCounter(input() for _ in range(int(input())))	Input and Counting: ·int(input()): Reads the number of words to follow. ·input() for _ in range(...): A generator expression that prompts the user to input a word for each count. ·OrderedCounter(...): Initializes the custom counter with the stream of input words. The resulting dictionary d stores {word: count} in the order of first appearance.

Line	Code	Explanation
5	<code>print(len(d))</code>	Prints the length of the OrderedCounter (which is the number of unique words).
6	<code>print(*d.values())</code>	<code>d.values()</code> : Gets a view object containing only the frequency counts (e.g., [1, 2, 1]). <code>*</code> : The splat operator (or asterisk) unpacks the list of values into arguments for the print function. This prints the numbers separated by a single space (the default separator for print).

Desired Input/Output Example:

- **Input (Count):** 4
- **Input (Words):** bcdef, abcdefg, bcdef, bcdef
- **Unique Words:** 2 (bcdef, abcdefg)
- **Word Frequencies in order of first appearance:**
 - bcdef: 3 times
 - abcdefg: 1 time
- **Output:**

```
2
3 1
```

5. Find the runner up score

This program finds and prints the **second highest unique score** (the runner-up score) from a list of scores.

Line	Code	Explanation
1	<code>if __name__ == '__main__':</code>	Standard execution block.
2	<code>n = int(input())</code>	Reads the count <i>n</i> (not strictly needed, but included in the original problem structure).
3	<code>arr = map(int, input().split())</code>	Reads the line of space-separated scores and creates a map object that yields integers.
4	<code>arr = set(arr)</code>	Converts the map (or list) of scores into a set . This automatically removes all duplicate scores, leaving only unique values.

Line	Code	Explanation
5	<code>arr.remove(max(arr))</code>	Finds the maximum score in the set and removes it. This leaves the set containing all scores <i>except</i> the highest one.
6	<code>print(max(arr))</code>	Finds the new maximum score in the remaining set. Since the previous maximum was removed, this new maximum is the second highest unique score (the runner-up).

Desired Input/Output Example:

- **Input (n):** 5
 - **Input (Scores):** 2 3 6 6 5
 - **Unique Scores (set):** 2,3,5,6
 - **Max Score:** 6
 - **Set after removing Max:** 2,3,5
 - **New Max (Runner-up):** 5
 - **Output:** 5
-